

MÓDULO 1 · AULAS 01 E 02

HTML & CSS

Fundamentos, Layout e Referência Completa

Apostila de apoio da Plataforma de Avaliações — o projeto fullstack construído ao longo do curso. Este material cobre o conteúdo das aulas 01 e 02 em profundidade, e vai além: Emmet, dicionários de HTML e CSS, e uma cola rápida para consulta.

Curso Fullstack · HTML/CSS · JavaScript · React · Node.js

Sumário

1. Como usar esta apostila

2. HTML semântico — a base de tudo

3. Emmet — escrevendo HTML na velocidade do pensamento

4. CSS — fundamentos e o box model

5. Seletores CSS e especificidade

6. Flexbox em profundidade (Aula 01)

7. CSS Grid em profundidade (Aula 02)

8. Responsividade e Mobile-First

9. Dicionário de tags HTML

10. Dicionário de propriedades CSS

11. Cola rápida (cheat sheet)

12. Glossário

13. Para praticar mais

1. Como usar esta apostila

Este material acompanha as **aulas 01 e 02** do curso, mas não se limita a elas. A ideia é dupla: (1) aprofundar os conceitos que você viu em sala, com mais contexto e exemplos do que cabe numa aula de 90 minutos, e (2) servir como **referência** para consultar sempre que precisar — inclusive nas próximas aulas, quando HTML e CSS deixam de ser o assunto principal mas continuam sendo usados o tempo todo.

Ao longo do texto você vai encontrar três tipos de caixa:

Dica

Um atalho, boa prática ou jeito mais rápido de fazer algo.

Atenção

Um erro comum ou uma pegadinha que costuma confundir quem está aprendendo.

No projeto

Como o conceito aparece de verdade no código da Plataforma de Avaliações.

Os dicionários (capítulos 9 e 10) e a cola rápida (capítulo 11) não precisam ser lidos de uma vez — eles existem para você abrir quando esquecer a sintaxe de alguma coisa.

2. HTML semântico — a base de tudo

Semântica, em HTML, significa escolher a tag pelo *significado* do conteúdo, não pela aparência visual. Um botão é uma tag `<button>` porque é clicável e aciona uma ação — não porque "parece um botão". Isso importa por três motivos concretos:

- **Acessibilidade:** leitores de tela (usados por pessoas com deficiência visual) anunciam "navegação", "botão", "cabeçalho" com base na tag usada — não no CSS aplicado.
- **SEO:** buscadores como o Google usam a estrutura semântica para entender do que a página trata.
- **Manutenibilidade:** um HTML com `<header>`, `<nav>`, `<main>` se lê como um índice do próprio código — um HTML feito só de `<div>` s exige adivinhação.

As tags estruturais principais

Tag	Quando usar
<code><header></code>	Cabeçalho da página ou de uma seção — geralmente logo, título e navegação.
<code><nav></code>	Um bloco de links de navegação principal (menu do site).
<code><main></code>	O conteúdo principal e único da página. Só pode existir um por página.
<code><section></code>	Um agrupamento temático de conteúdo, geralmente com seu próprio título (h2/h3).
<code><article></code>	Conteúdo independente que faria sentido sozinho, fora do contexto da página (um post, um card de lugar, um comentário).
<code><aside></code>	Conteúdo relacionado, mas secundário — uma barra lateral, um resumo, um destaque.
<code><footer></code>	Rodapé da página ou de uma seção — créditos, links secundários, copyright.

No projeto

O `index.html` da aula 01 usa exatamente essa estrutura: `<header class="cabecalho">` com o logo e a navegação, `<main class="container">` envolvendo o conteúdo, e cada card de lugar é um `<article class="card">` — porque cada card faz sentido "sozinho" (poderia ser compartilhado isoladamente e ainda faria sentido).

Um exemplo lado a lado

Compare a mesma estrutura visual escrita sem e com semântica:

Sem semântica (funciona, mas não diz nada sobre o conteúdo):

```
<div class="topo">
  <div class="logo">Avaliações</div>
  <div class="menu"><div>Lugares</div></div>
</div>
<div class="conteudo">
  <div class="item">Café Aroma</div>
</div>
```

Com semântica (o próprio HTML documenta a intenção):

```
<header>
  <a class="logo">Avaliações</a>
  <nav><a href="#">Lugares</a></nav>
</header>
<main>
  <article class="card">Café Aroma</article>
</main>
```

 Dica

Regra prática: se você não sabe qual tag usar, pergunte "o que este bloco é?", não "como ele deve parecer?". A aparência é trabalho do CSS.

3. Emmet — escrevendo HTML na velocidade do pensamento

Emmet é um "atalho de digitação" embutido no VS Code (e na maioria dos editores modernos) por padrão — nenhuma instalação necessária. Você digita uma abreviação curta, aperta **Tab** (ou Enter), e ela se expande em HTML completo. Depois de acostumado, é comum escrever uma tela inteira de HTML em segundos.

 Dica

No VS Code, crie um arquivo `.html`, digite qualquer abreviação desta seção e aperte Tab. Se nada acontecer, confira se a extensão "Emmet" está habilitada (ela vem ativada por padrão).

Sintaxe básica

Símbolo	Significado	Você digita	Emmet expande para
>	filho (aninha dentro)	nav>ul>li	<nav> </nav>
+	irmão (lado a lado)	h1+p	
*	multiplicação	li*3	três repetidos
.	classe	div.card	<div class="card"></div>
#	id	div#app	<div id="app"></div>
{ }	texto interno	a{Ver detalhes}	<a>Ver detalhes
[]	atributo	a[href="#" target="_blank"]	
()	agrupamento	(header>h1)+main	agrupa header>h1 antes de anexar main como irmão
\$	numeração automática	li.item\$*3	item1, item2, item3 nas classes
^	sobe um nível (sai do aninhamento)	div>p^div	div com p dentro, seguido de outro div irmão do primeiro

Exemplos aplicados ao projeto

Estas abreviações geram estruturas parecidas com o que você escreveu (ou vai escrever) na Plataforma de Avaliações:

Você digita	O que gera
header.cabecalho>a.cabecalho__logo+nav.cabecalho__nav>a	Um cabeçalho com logo e navegação — a estrutura exata do topo de index.html .
article.card>div.card-banner+div.card-body>span.badge+h2.card-title+p.card-cidade+p.card-descricao+a.card-link	Um card de lugar completo, com todos os elementos internos, em uma linha.
div.cards-grid>article.card*6	A grade inteira: um container com 6 cards vazios

	dentro, prontos para preencher.
<code>ul.lista-avaliacoes>li.avaliacao-item*3>div.avaliacao-cabecalho>strong+span.estrelas</code>	Três itens de avaliação, cada um já com o cabeçalho (nome + estrelas) montado.

Emmet também funciona em CSS

Dentro de um arquivo `.css`, as abreviações mudam de propósito, mas o efeito é o mesmo: menos digitação.

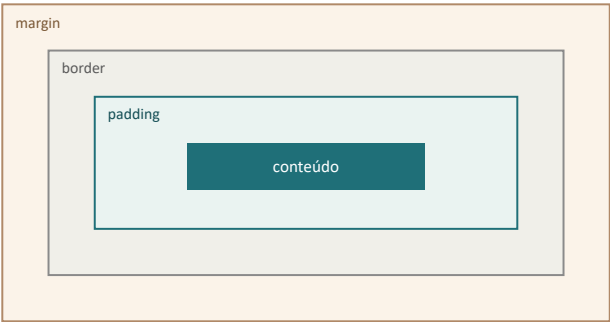
Você digita	Expande para
<code>d:f</code>	<code>display: flex;</code>
<code>jc:c</code>	<code>justify-content: center;</code>
<code>ai:c</code>	<code>align-items: center;</code>
<code>m10</code>	<code>margin: 10px;</code>
<code>p0</code>	<code>padding: 0;</code>
<code>w100p</code>	<code>width: 100%;</code>
<code>bdrs4</code>	<code>border-radius: 4px;</code>
<code>pos:r</code>	<code>position: relative;</code>

⚠️ Atenção

Emmet é um acelerador de digitação, não um substituto para entender o que você está escrevendo. Se você não souber explicar o HTML/CSS que o Emmet gerou, volte um passo e estude o conceito antes de continuar.

4. CSS — fundamentos e o box model

Todo elemento HTML é, para o navegador, uma caixa retangular. O **box model** descreve as quatro camadas dessa caixa, de dentro para fora:



As quatro camadas de qualquer elemento: conteúdo, padding (respiro interno), border e margin (respiro externo).

Camada	O que é	Propriedade CSS
Conteúdo	O texto ou elementos filhos.	<code>width</code> , <code>height</code>
Padding	Espaço interno, entre o conteúdo e a borda.	<code>padding</code>
Border	A borda visível (ou invisível) da caixa.	<code>border</code>
Margin	Espaço externo, entre esta caixa e as vizinhas.	<code>margin</code>

box-sizing : o detalhe que evita 90% dos problemas de layout

Por padrão (`box-sizing: content-box`), quando você define `width: 200px` , o navegador soma padding e border *por cima* desse valor — uma caixa com `width: 200px` e `padding: 20px` ocupa, na prática, 240px. Isso é contraintuitivo e causa layouts que "estouram" o espaço disponível.

Por isso, praticamente todo projeto CSS moderno começa com esta regra, que faz o `width / height` já *incluir* padding e border:

```
* {
  box-sizing: border-box;
}
```

No projeto

O `css/style.css` das aulas 01-02 começa exatamente assim — é o primeiro bloco de código de todo o CSS do curso, por um motivo: sem ele, cada cálculo de largura de card na grade Flexbox/Grid ficaria mais complicado.

5. Seletores CSS e especificidade

Tipos de seletor

Seletor	Exemplo	Seleciona
Elemento	<code>p</code>	Todos os <code><p></code> .
Classe	<code>.card</code>	Todo elemento com <code>class="card"</code> (pode repetir).
Id	<code>#app</code>	O único elemento com <code>id="app"</code> .
Atributo	<code>[data-categoria="museu"]</code>	Elementos com esse atributo/valor.
Descendente (espaço)	<code>.card p</code>	Qualquer <code><p></code> dentro de <code>.card</code> , em qualquer profundidade.
Filho direto (<code>></code>)	<code>.cards-grid > .card</code>	Só os <code>.card</code> filhos <i>diretos</i> de <code>.cards-grid</code> .
Pseudo-classe	<code>.card:hover</code>	O elemento quando o mouse está sobre ele.
Pseudo-classe estrutural	<code>li:first-child</code>	O primeiro <code></code> dentro do pai.
Pseudo-elemento	<code>.card::before</code>	Um elemento "virtual" inserido antes do conteúdo.

Especificidade — quem vence quando duas regras se aplicam?

Quando mais de uma regra CSS mira o mesmo elemento, o navegador decide qual usar por **especificidade** — uma pontuação, do mais forte para o mais fraco:

1. `id` (ex: `#app`) — mais específico
2. `classe`, `atributo`, `pseudo-classe` (ex: `.card`, `:hover`)
3. `elemento`, `pseudo-elemento` (ex: `div`, `::before`) — menos específico

Em caso de empate na especificidade, **a regra que vem depois no arquivo CSS vence** — essa ordem de leitura de cima para baixo é a "cascata" que dá nome ao CSS (*Cascading* Style Sheets).

 Atenção

Evite `!important`. Ele quebra a cascata normal e força uma regra a vencer não importa o quê — o que torna o CSS muito mais difícil de depurar depois. Se você "precisa" de `!important`, quase sempre há um seletor mais específico (ou uma reorganização do CSS) que resolve o problema de forma mais sustentável.

6. Flexbox em profundidade (Aula 01)

Flexbox organiza elementos em **uma dimensão** — uma linha ou uma coluna — e é a ferramenta certa quando você precisa distribuir, alinhar ou permitir que itens cresçam/encolham dentro de um espaço disponível. É exatamente o problema da grade de cards da aula 01: uma lista de itens que deve se reorganizar sozinha conforme o espaço da tela.

Dois eixos, um container, muitos itens

Ao aplicar `display: flex` a um elemento, ele vira um **flex container**, e todos os filhos diretos viram **flex items**. O container ganha dois eixos:

- **Eixo principal (main axis):** a direção em que os itens fluem — horizontal por padrão (`flex-direction: row`).
- **Eixo cruzado (cross axis):** perpendicular ao principal — vertical, se o principal for horizontal.

Dica

`justify-content` sempre atua no **eixo principal**, e `align-items` sempre no **eixo cruzado**. Se você trocar `flex-direction` para `column`, os dois eixos invertem — e o que "justify" e "align" controlam, inverte junto.

Propriedades do container

Propriedade	O que controla	Valores comuns
<code>display</code>	Ativa o Flexbox.	<code>flex</code>
<code>flex-direction</code>	Direção do eixo principal.	<code>row</code> (padrão), <code>column</code> , <code>row-reverse</code> , <code>column-reverse</code>
<code>flex-wrap</code>	Se os itens quebram linha quando não cabem.	<code>nowrap</code> (padrão), <code>wrap</code>
<code>justify-content</code>	Alinhamento no eixo principal.	<code>flex-start</code> , <code>center</code> , <code>space-between</code> , <code>space-around</code> , <code>space-evenly</code>
<code>align-items</code>	Alinhamento no eixo cruzado.	<code>stretch</code> (padrão), <code>flex-start</code> , <code>center</code> , <code>flex-end</code> , <code>baseline</code>
<code>align-content</code>	Alinhamento de múltiplas linhas (só com <code>wrap</code>).	mesmos valores de <code>justify-content</code>
<code>gap</code>	Espaçamento entre itens (substitui <code>margin</code> manual).	<code>16px</code> , <code>1rem</code> , <code>2rem</code>

Propriedades dos itens

Propriedade	O que controla
<code>flex-grow</code>	O quanto o item cresce para ocupar espaço extra (0 = não cresce).
<code>flex-shrink</code>	O quanto o item encolhe quando falta espaço (1 = encolhe normalmente).
<code>flex-basis</code>	O tamanho "ideal" do item antes de crescer/encolher.
<code>flex</code>	Atalho para as três acima: <code>flex: 1 1 280px;</code>
<code>align-self</code>	Sobrescreve <code>align-items</code> para um item específico.
<code>order</code>	Muda a ordem visual sem mudar o HTML.

No projeto

A grade de cards usa exatamente esta combinação:

```
.cards-grid {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 24px;  
}  
.card {  
  flex: 1 1 280px;  
  max-width: 320px;  
}
```

Tradução: "quebre linha quando precisar, com 24px de respiro entre os cards; cada card tenta ter 280px, pode crescer para preencher a linha, mas nunca passa de 320px."

Erros comuns

- **Esquecer `flex-wrap: wrap`** : sem isso, os itens tentam caber todos numa linha só e encolhem até ficarem espremidos, em vez de quebrar linha.
- **Confundir `justify-content` com `align-items`** : lembre-se qual eixo é o principal antes de escolher qual propriedade usar.
- **Achar que `flex-basis` é o mesmo que `width`** : são parecidos, mas `flex-basis` é só o ponto de partida — `flex-grow` / `flex-shrink` ainda podem alterar o tamanho final.

7. CSS Grid em profundidade (Aula 02)

Enquanto Flexbox pensa em **uma dimensão**, CSS Grid pensa em **duas ao mesmo tempo** — linhas e colunas simultaneamente. É a ferramenta certa para layouts de página inteira, como a página de detalhe da aula 02, que precisa organizar informações em regiões (principal, resumo, avaliações) de forma bidimensional.

Propriedades do container

Propriedade	O que controla
<code>display: grid</code>	Ativa o CSS Grid.
<code>grid-template-columns</code>	Define as colunas (quantas e o tamanho de cada uma).
<code>grid-template-rows</code>	Define as linhas.
<code>grid-template-areas</code>	Nomeia regiões do grid, para associar itens por nome em vez de número.
<code>gap</code>	Espaçamento entre linhas e colunas.
<code>justify-items</code> / <code>align-items</code>	Alinhamento do conteúdo dentro de cada célula (horizontal/vertical).

Propriedades dos itens

Propriedade	O que controla
<code>grid-column</code>	Em quais colunas o item começa/termina (ex: <code>1 / 3</code>).
<code>grid-row</code>	Em quais linhas o item começa/termina.
<code>grid-area</code>	Associa o item a um nome definido em <code>grid-template-areas</code> .

Unidades e funções úteis

Sintaxe	Significado
<code>fr</code>	"Fração" do espaço disponível — <code>1fr 2fr</code> divide em 1/3 e 2/3.
<code>repeat(3, 1fr)</code>	Atalho para <code>1fr 1fr 1fr</code> .
<code>minmax(200px, 1fr)</code>	Nunca menor que 200px, mas cresce até 1fr se houver espaço.
<code>repeat(auto-fit, minmax(200px, 1fr))</code>	Quantas colunas de pelo menos 200px couberem, se ajustando automaticamente.

No projeto

A página de detalhe usa `grid-template-areas` para nomear as três regiões e reorganizá-las só a partir de 768px:

```
@media (min-width: 768px) {  
  .detalhe {  
    grid-template-columns: 2fr 1fr;  
    grid-template-areas:  
      "principal resumo"  
      "avaliacoes avaliacoes";  
  }  
  .detalhe-principal { grid-area: principal; }  
  .detalhe-resumo     { grid-area: resumo; }  
  .detalhe-avaliacoes { grid-area: avaliacoes; }  
}
```

Repare que "avaliacoes" aparece duas vezes na mesma linha do `grid-template-areas` — isso é o que faz aquela região ocupar as duas colunas (largura total).

Grid ou Flexbox — qual escolher?

Use Flexbox quando...	Use Grid quando...
O layout é uma lista/fluxo em uma dimensão.	Você precisa controlar linhas <i>e</i> colunas ao mesmo tempo.
Você não sabe quantos itens vai ter (uma grade de cards que cresce).	Você sabe a estrutura da página (cabecalho, sidebar, conteúdo, rodapé).
Você quer que os itens "empurrem" uns aos outros naturalmente.	Você quer nomear regiões e posicionar itens de forma precisa.

Na prática, os dois convivem no mesmo projeto — como você já viu: Flexbox para a grade de cards, Grid para a página de detalhe.

8. Responsividade e Mobile-First

Mobile-first é uma forma de pensar, antes de ser uma técnica: você escreve o CSS *base* pensando na tela pequena (sem nenhuma media query), e só adiciona complexidade — mais colunas, mais espaçamento — conforme a tela cresce, usando `@media (min-width: ...)`.

Dica

O oposto (desktop-first) parte do design grande e vai "desmontando" com `max-width`. Mobile-first costuma gerar CSS mais enxuto, porque o caso mais simples (empilhado) não precisa de nenhuma regra especial — é o comportamento padrão do HTML.

Sintaxe de media query

```
/* Regras base: valem para QUALQUER tamanho de tela */
.detalhe {
  display: grid;
  gap: 24px;
}

/* A partir de 768px, adiciona/sobrescreve regras */
@media (min-width: 768px) {
  .detalhe {
    grid-template-columns: 2fr 1fr;
  }
}
```

Breakpoints comuns

Largura	Dispositivo típico
576px	Celulares em paisagem
768px	Tablets
992px	Notebooks pequenos
1200px	Desktops

Esses valores não são regras fixas do CSS — são convenções. O critério certo é sempre "onde o *seu* layout específico começa a parecer ruim?", não um número mágico.

Unidades relativas

Unidade	Relativa a...
%	O tamanho do elemento pai.
vw / vh	1% da largura/altura da <i>viewport</i> (janela visível).
em	O tamanho de fonte do elemento pai.
rem	O tamanho de fonte do elemento raiz (<code><html></code>) — não composto, mais previsível.

Dica

Prefira `rem` para tamanhos de fonte. Como `em` é relativo ao pai, tamanhos em `em` "compõem" (multiplicam) conforme o aninhamento aumenta, o que causa tamanhos inesperados em estruturas profundas. `rem` sempre parte da mesma referência (a raiz do documento).

 Atenção

Sem esta linha no `<head>` , o navegador do celular simula uma tela larga e nenhuma media query funciona como esperado:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

9. Dicionário de tags HTML

Estrutura do documento

Tag	Descrição
<!DOCTYPE html>	Declara que o documento é HTML5. Sempre a primeira linha.
<html>	Elemento raiz do documento.
<head>	Metadados: título, links de CSS, configurações — nada visível.
<meta>	Metadado (charset, viewport, descrição para SEO).
<title>	Título da aba do navegador.
<link>	Vincula um recurso externo (geralmente CSS ou ícone).
<style>	CSS embutido diretamente na página.
<script>	JavaScript embutido ou vinculado (src="...").
<body>	Todo o conteúdo visível da página.

Estrutura semântica

Tag	Descrição
<header>	Cabeçalho da página ou de uma seção.
<nav>	Bloco de navegação principal.
<main>	Conteúdo principal (único por página).
<section>	Agrupamento temático de conteúdo.
<article>	Conteúdo independente e autocontido.
<aside>	Conteúdo secundário/relacionado.
<footer>	Rodapé da página ou de uma seção.
<div>	Bloco genérico, sem significado semântico — use quando nenhuma tag específica se aplica.
	Igual ao div , mas em linha (inline) — para trechos dentro de um texto.

Texto

Tag	Descrição
<h1> a <h6>	Títulos, do mais importante (h1) ao menos importante (h6). Não pule níveis por estética.
<p>	Parágrafo de texto.
<a>	Link (âncora) — atributo href define o destino.
	Texto com importância forte (visualmente negrito, semanticamente relevante).
	Texto com ênfase (visualmente itálico).
 	Quebra de linha (use com moderação — geralmente CSS resolve melhor).
<hr>	Linha horizontal, separando conteúdos.
/ /	Lista não ordenada / ordenada / item de lista.
<code>	Trecho de código, em fonte monoespaçada.

Formulário

Tag	Descrição
<form>	Envolve os campos de um formulário.
<label>	Rótulo de um campo — associe com <code>for</code> = <code>id</code> do campo.
<input>	Campo de entrada — o atributo <code>type</code> muda o comportamento (<code>text</code> , <code>email</code> , <code>password</code> , <code>number</code> ...).
<textarea>	Campo de texto de múltiplas linhas.
<select> / <option>	Lista suspensa e suas opções.
<button>	Botão clicável — <code>type="submit"</code> envia o formulário.

Mídia e tabelas

Tag	Descrição
	Imagem — sempre defina <code>alt</code> (texto alternativo, essencial para acessibilidade).
<svg>	Gráfico vetorial desenhado diretamente no HTML.
<video> / <audio>	Reprodução de vídeo/áudio nativo do navegador.
<table>	Tabela de dados (não use para layout — isso é papel do CSS).
<thead> / <tbody>	Agrupa o cabeçalho e o corpo da tabela.
<tr> / <th> / <td>	Linha / célula de cabeçalho / célula de dado.

10. Dicionário de propriedades CSS

Layout e posicionamento

Propriedade	Descrição
display	Como o elemento se comporta no layout (block , inline , flex , grid , none).
position	static (padrão), relative , absolute , fixed , sticky .
top / right / bottom / left	Deslocamento, usado junto com position .
z-index	Ordem de empilhamento (quem fica na frente de quem).
overflow	O que fazer quando o conteúdo não cabe (visible , hidden , scroll , auto).

Box model

Propriedade	Descrição
width / height	Largura/altura do conteúdo (ou da caixa toda, com border-box).
margin	Espaço externo. Aceita 1 a 4 valores (topo, direita, baixo, esquerda).
padding	Espaço interno. Mesma sintaxe de valores do margin .
border	Atalho para border-width , border-style , border-color .
box-sizing	content-box (padrão) ou border-box (recomendado).

Flexbox e Grid

Ver tabelas completas nos capítulos 6 e 7.

Tipografia

Propriedade	Descrição
font-family	Lista de fontes, em ordem de preferência.
font-size	Tamanho do texto.
font-weight	Peso da fonte (400 normal, 700 negrito).
line-height	Altura da linha — controla o "respiro" vertical do texto.
text-align	Alinhamento horizontal do texto.
text-decoration	Sublinhado, tachado etc. (none remove o sublinhado padrão de links).
letter-spacing	Espaçamento entre letras.

Cor e fundo

Propriedade	Descrição
color	Cor do texto.
background-color	Cor de fundo.
background-image	Imagem de fundo.
opacity	Transparência do elemento inteiro (0 a 1).

Efeitos e transições

Propriedade	Descrição
<code>border-radius</code>	Arredonda os cantos.
<code>box-shadow</code>	Sombra projetada pela caixa.
<code>transition</code>	Anima mudanças de propriedades (ex: cor no hover).
<code>transform</code>	Move, escala, rotaciona o elemento sem afetar o layout ao redor.
<code>cursor</code>	Ícone do mouse ao passar por cima (<code>pointer</code> para indicar clicável).

11. Cola rápida (cheat sheet)

Uma página condensada para consulta rápida enquanto você programa.

Emmet essencial

<code>.card*3</code>	3 divs
<code>ul>li*3</code>	lista com 3 itens
<code>a{Texto}</code>	link com texto
<code>a[href="#"]</code>	link com atributo
<code>.item\$*3</code>	item1, item2, item3

Grid essencial

<code>display: grid;</code>	ativa
<code>grid-template-columns: 1fr 1fr;</code>	2 colunas iguais
<code>repeat(3, 1fr)</code>	3 colunas iguais
<code>grid-area: nome;</code>	posiciona por nome
<code>gap: 16px;</code>	espaçamento

Flexbox essencial

<code>display: flex;</code>	ativa
<code>flex-wrap: wrap;</code>	quebra linha
<code>justify-content: center;</code>	eixo principal
<code>align-items: center;</code>	eixo cruzado
<code>flex: 1 1 280px;</code>	grow shrink basis

Media query essencial

```
/* mobile-first */
.algo { }

@media (min-width: 768px) {
  .algo { }
}
```

12. Glossário

Semântica

Escolher a tag HTML pelo significado do conteúdo, não pela aparência.

DOM (Document Object Model)

A representação do HTML em memória, que o navegador usa para desenhar a página e que o JavaScript pode manipular.

Cascata

A regra que decide qual estilo CSS "vence" quando há conflito: especificidade primeiro, ordem no arquivo depois.

Especificidade

A "pontuação" de um seletor CSS — id > classe > elemento.

Responsivo

Um layout que se adapta a diferentes tamanhos de tela.

Viewport

A área visível da página no dispositivo do usuário.

Box model

O modelo de conteúdo + padding + border + margin que descreve toda caixa HTML.

Container

Um elemento pai que aplica um contexto de layout (flex container, grid container) aos seus filhos diretos.

Breakpoint

A largura de tela onde uma media query muda o layout.

Wireframe

Um esboço de baixa fidelidade da estrutura de uma tela, sem se preocupar com cores ou detalhes visuais finais.

Mobile-first

Estratégia de escrever CSS base para telas pequenas e adicionar complexidade para telas maiores.

Acessibilidade

Tornar uma interface utilizável por pessoas com diferentes capacidades, incluindo quem usa leitores de tela.

13. Para praticar mais

- **Flexbox Froggy** e **CSS Grid Garden** — jogos gratuitos, no navegador, que ensinam Flexbox e Grid resolvendo puzzles visuais.
- **Recrie um layout real:** escolha um site que você usa bastante, abra o DevTools (F12) e tente recriar um pedaço dele (um cabeçalho, uma grade de produtos) do zero.
- **Desafio Emmet:** construa a mesma estrutura duas vezes — uma digitando tag por tag, outra só com abreviações Emmet — e compare o tempo.
- **Inspecione o CSS de terceiros:** use o DevTools para inspecionar elementos de sites reais e identificar se usam Flexbox, Grid, ou os dois.
- **Releia o próprio projeto:** volte no `css/style.css` das aulas 01-02 depois de ler esta apostila — você vai reconhecer cada propriedade e entender o "porquê" por trás de cada escolha.